ĐẶC TẢ USE-CASE TRỌNG TÂM

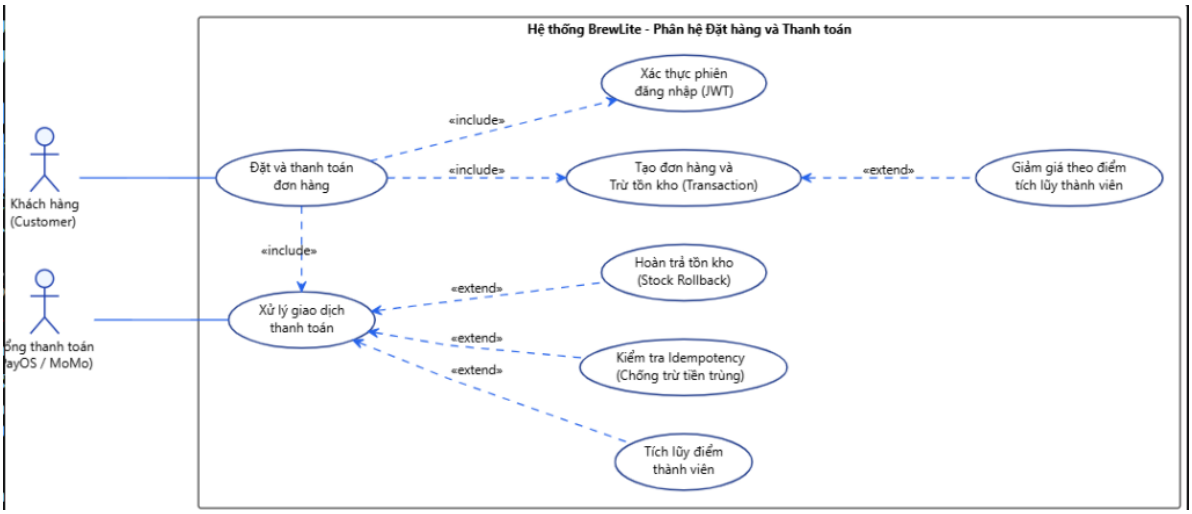
UC-01: ĐẶT VÀ THANH TOÁN ĐƠN HÀNG

Dự án: Hệ thống Đặt đồ uống trực tuyến BrewLite
Học phần: Công nghệ Phần mềm (Software Engineering) – Assignment 2
Mã nhiệm vụ: Task 1.3 **Mã Use-case:** UC-01
Phiên bản: 1.0 **Ngày lập:** 04/11/2026
Tệp nguồn PlantUML: `docs/diagrams/usecase-checkout.puml`

1 Giới thiệu & Bối cảnh nghiệp vụ

Trong hệ sinh thái ứng dụng đặt đồ uống trực tuyến BrewLite, nghiệp vụ **"Đặt và thanh toán đơn hàng"** giữ vai trò sống còn (Core Transactional Value). Use-case này kết nối trực tiếp trải nghiệm lựa chọn thức uống của khách hàng (Client Next.js PWA) với hệ thống xử lý giao dịch máy chủ (NestJS API), đảm bảo trừ tồn kho an toàn thông qua cơ chế Khóa lạc quan (Optimistic Concurrency Control), khởi tạo đơn hàng trong cơ chế giao dịch ACID toàn vẹn, và thanh toán chống trừ tiền trùng lặp (Idempotency Key).

2 Sơ đồ Usecase (Use-Case Diagram)



Hình 1: Sơ đồ Use-case Đặt và thanh toán đơn hàng

Cấu trúc quan hệ nghiệp vụ trong sơ đồ:

1. **Use-case trọng tâm:** `Đặt và thanh toán đơn hàng` (Place Order & Pay)
2. **Các Use-case thành phần bắt buộc (`include`):**
   - `Xác thực phiên đăng nhập (JWT)`: Kiểm tra tính hợp lệ của token trước khi đặt đơn.
   - `Tạo đơn hàng và Trừ tồn kho (Transaction)`: Thực thi trong 1 Database Transaction an toàn.
   - `Xử lý giao dịch thanh toán`: Tương tác với Cổng thanh toán (Payment Service) để trừ tiền trong tài khoản.
3. **Các Use-case mở rộng theo điều kiện (`extend`):**
   - `Giảm giá theo điểm tích lũy thành viên`: `Đặt và thanh toán đơn hàng` (khi khách đạt hạng Bạc, Vàng, Kim Cương ≥ 20 điểm).
   - `Tích lũy điểm thành viên`: `Xử lý giao dịch thanh toán` (khi thanh toán thành công chuyển sang `PAID`).
   - `Kiểm tra Idempotency (Chống trừ tiền trùng)`: `Xử lý giao dịch thanh toán` (khi client gửi lại request cùng key).
   - `Hoàn trả tồn kho (Stock Rollback)`: `Xử lý giao dịch thanh toán` (khi giao dịch bị từ chối / thất bại).

3 Đặc tả chi tiết từng Use-Case

3.1 UC-01: Đặt và thanh toán đơn hàng (Use-Case Tổng hợp Trọng tâm)

Bảng 1: Bảng đặc tả Use-Case chi tiết UC-01: Đặt và thanh toán đơn hàng

Usecase ID: UC-01 – Đặt và thanh toán đơn hàng	
Tóm tắt: Chức năng cho phép Khách hàng kiểm tra giỏ hàng, áp dụng giảm giá dựa trên điểm tích lũy thành viên, kiểm tra và trừ tồn kho bằng Database Transaction an toàn, thực hiện thanh toán qua Cổng thanh toán (hỗ trợ Idempotency chống trừ tiền trùng) và tích lũy điểm thưởng thành viên khi hoàn tất.	
Dòng sự kiện chính	1. Khách hàng (Customer) truy cập giỏ hàng trên Giao diện người dùng (Next.js UI) và nhấn nút **"Tiến hành đặt hàng"**. 2. Next.js UI gửi yêu cầu xác thực phiên đăng nhập (JWT Bearer Token) đến Hệ thống Backend (NestJS API). 3. NestJS API xác thực tính hợp lệ của Token, trích xuất mã người dùng (`userId`) cùng số điểm tích lũy thành viên (`loyaltyPoints`) hiện có. 4. Next.js UI tính toán chiết khấu giảm giá thành viên, hiển thị tổng tiền cuối cùng và Khách hàng nhấn **"Xác nhận thanh toán"**. 5. Next.js UI sinh mã khóa Idempotency độc nhất (`idempotencyKey`) và gửi yêu cầu tạo đơn hàng `POST /orders` kèm chi tiết giỏ hàng đến NestJS API. 6. NestJS API mở Transaction (`$transaction`) trên Cơ sở dữ liệu (PostgreSQL): kiểm tra tồn kho (`stock >= quantity`), trừ kho theo khóa lạc quan (`version`), áp dụng giảm giá điểm tích lũy và lưu bản ghi Đơn hàng với trạng thái `PENDING`. 7. Next.js UI nhận phản hồi tạo đơn thành công, tiếp tục gửi yêu cầu xử lý thanh toán `POST /payments` kèm `orderId`, phương thức thanh toán và `idempotencyKey` đến NestJS API. 8. NestJS API kiểm tra khóa `idempotencyKey` trong CSDL chống trùng lặp, sau đó chuyển tiếp yêu cầu giao dịch sang Cổng thanh toán (PayOS / MoMo). 9. Cổng thanh toán xử lý trừ tiền từ tài khoản/ví của khách hàng và phản hồi trạng thái `SUCCESS` kèm mã giao dịch (`transactionId`) về cho NestJS API. 10. NestJS API mở Database Transaction: cập nhật trạng thái đơn hàng sang `PAID`, lưu bản ghi thanh toán, tự động cộng điểm tích lũy thành viên cho khách hàng, và phản hồi HTTP `200 OK`. 11. Next.js UI nhận kết quả thành công, thực hiện xóa sạch giỏ hàng (`clearCart()`) và điều hướng khách hàng đến màn hình chi tiết theo dõi đơn hàng (`/orders/:id`).

(Xem tiếp trang sau...)

Usecase ID: UC-01 – Đặt và thanh toán đơn hàng (tiếp theo)	
Dòng sự kiện phụ	• **2a. Phiên đăng nhập hết hạn hoặc chưa đăng nhập:** Next.js UI lưu trạng thái giỏ hàng và chuyển hướng người dùng đến trang Đăng nhập (`/login?redirect=/checkout`). • **4a. Khách hàng chưa đạt mốc tích lũy điểm:** Hệ thống không áp dụng chiết khấu, giữ nguyên giá gốc của đơn hàng. • **6a. Sản phẩm hết hàng hoặc xung đột tồn kho (Optimistic Lock):** NestJS API hủy bỏ Transaction (Rollback 100%) và trả về HTTP 400/409; Next.js UI hiển thị thông báo lỗi để người dùng điều chỉnh lại giỏ hàng. • **8a. Gửi lại request với Idempotency Key đã tồn tại:** NestJS API phát hiện khóa đã được xử lý thành công, lập tức trả về kết quả cũ kèm HTTP 200 mà không gọi Cổng thanh toán lần 2 (ngăn chặn trừ tiền trùng). • **9a. Cổng thanh toán từ chối hoặc giao dịch thất bại:** Cổng thanh toán trả về `FAILED`; NestJS API cập nhật trạng thái đơn thành `PAYMENT_FAILED`, kích hoạt cơ chế hoàn trả số lượng kho đã trừ (Stock Rollback) và báo lỗi về UI để khách thử lại phương thức khác. • **9b. Cổng thanh toán mất kết nối hoặc quá thời gian (Timeout):** NestJS API giữ đơn hàng ở trạng thái `PENDING` để phục vụ cơ chế đối soát hoặc kiểm tra lại sau.
Tiền điều kiện	• Khách hàng đã đăng nhập vào hệ thống và sở hữu JSON Web Token (JWT Bearer) còn hiệu lực. • Giỏ hàng có ít nhất một sản phẩm hợp lệ (`items.length > 0`). • Toàn bộ sản phẩm và topping trong đơn hàng đều đang được mở bán (`isAvailable = true`).
Hậu điều kiện	• **Hậu điều kiện thành công (Success):** Đơn hàng được lưu vào CSDL với trạng thái `PAID`; tồn kho bị trừ tương ứng và trường `version` tăng 1; giao dịch thanh toán được ghi nhận cùng `idempotencyKey`; khách hàng được cộng điểm tích lũy thành viên; giỏ hàng được làm rỗng và giao diện chuyển đến trang chi tiết đơn hàng. • **Hậu điều kiện thất bại (Failure):** Giao dịch bị Rollback hoặc được hoàn trả tồn kho (Stock Rollback); đơn hàng chuyển sang trạng thái `PAYMENT_FAILED` hoặc không được tạo; giỏ hàng của khách hàng được bảo lưu nguyên vẹn.

3.2 UC-02: Xác thực phiên đăng nhập (JWT)

Usecase ID: UC-02 – Xác thực phiên đăng nhập (JWT)	
Tóm tắt: Chức năng kiểm tra tính hợp lệ và thời hạn của JSON Web Token (JWT Bearer) trong request header nhằm xác thực danh tính khách hàng trước khi cho phép thực hiện các thao tác đặt hàng và thanh toán.	
Dòng sự kiện chính	1. Next.js UI lấy chuỗi Bearer Token từ bộ nhớ LocalStorage và đính kèm vào Header `Authorization: Bearer <token>` của HTTP request. 2. Next.js UI gửi request đến NestJS API. 3. `AuthGuard` tại NestJS API chặn request và trích xuất chuỗi token từ Header. 4. `JwtService` thực hiện giải mã (verify signature) bằng secret key của hệ thống và kiểm tra thời hạn sống (`exp`) của token. 5. Hệ thống trích xuất thông tin người dùng (`userId`, `role`, `email`) và gán vào context đối tượng `Request.user`. 6. NestJS API cho phép request tiếp tục đi vào controller xử lý đơn hàng.
Dòng sự kiện phụ	• **1a. Token không tồn tại (chưa đăng nhập):** Next.js UI chuyển hướng người dùng sang trang Đăng nhập (`/login?redirect=/checkout`). • **4a. Token không hợp lệ hoặc bị giả mạo:** NestJS API từ chối request và trả về mã lỗi HTTP 401 Unauthorized. • **4b. Token đã hết hạn (Expired Token):** NestJS API trả về HTTP 401; UI xóa token cũ trong LocalStorage và yêu cầu khách hàng đăng nhập lại.
Tiền điều kiện	Khách hàng thực hiện hành động yêu cầu định danh (xác nhận giỏ hàng, đặt hàng, gửi thanh toán).
Hậu điều kiện	• **Thành công:** Danh tính khách hàng được xác thực hợp lệ và gán vào request context, cho phép thực thi logic nghiệp vụ tiếp theo. • **Thất bại:** Request bị chặn lại với mã lỗi HTTP 401 Unauthorized, không có dữ liệu nghiệp vụ nào bị sửa đổi.

Bảng 2: Bảng đặc tả Use-Case UC-02: Xác thực phiên đăng nhập (JWT)

3.3 UC-03: Tạo đơn hàng và Trừ tồn kho (Transaction)

Usecase ID: UC-03 – Tạo đơn hàng và Trừ tồn kho (Transaction)	
Tóm tắt: Chức năng khởi tạo bản ghi đơn hàng ở trạng thái `PENDING` và đồng thời trừ số lượng tồn kho theo cơ chế Khóa lạc quan (Optimistic Concurrency Control qua trường `version`) trong một Atomic Database Transaction an toàn.	
Dòng sự kiện chính	1. Next.js UI gửi request `POST /orders` kèm chi tiết giỏ hàng và Bearer JWT Token đến NestJS API. 2. NestJS API kiểm tra trạng thái mở bán (`isAvailable = true`) của các sản phẩm và topping trong đơn hàng. 3. NestJS API mở Transaction nguyên tử trên PostgreSQL (`prisma.$transaction`). 4. Hệ thống truy vấn tồn kho hiện tại và số phiên bản `version` của từng sản phẩm trong giỏ hàng. 5. Hệ thống kiểm tra số lượng khả dụng: nếu `stock >= requestedQuantity`, thực hiện trừ tồn kho và tăng phiên bản (`stock = stock - qty`, `version = version + 1`). 6. Hệ thống tạo bản ghi `Order` với trạng thái `PENDING`, cùng các bản ghi con `OrderItem` và `OrderItemTopping`. 7. Database Transaction commit thành công; NestJS API phản hồi HTTP 201 Created kèm `orderId` và `totalAmount`.
Dòng sự kiện phụ	• **5a. Tồn kho không đủ (`stock < qty`):** Transaction tự động Rollback 100%, API trả về HTTP 400 Bad Request kèm thông báo sản phẩm đã hết hàng. • **5b. Xung đột ghi đồng thời (Optimistic Lock – sai `version`):** Transaction Rollback, trả về HTTP 409 Conflict yêu cầu khách hàng thử lại.
Tiền điều kiện	Khách hàng đã được xác thực danh tính (JWT hợp lệ); giỏ hàng có ít nhất một sản phẩm mở bán.
Hậu điều kiện	• **Thành công:** Đơn hàng mới được tạo ở trạng thái `PENDING`; tồn kho sản phẩm bị trừ chính xác và `version` tăng thêm 1. • **Thất bại:** CSDL giữ nguyên trạng thái ban đầu, không phát sinh đơn hàng rác và tồn kho không bị trừ âm.

Bảng 3: Bảng đặc tả Use-Case UC-03: Tạo đơn hàng và Trừ tồn kho (Transaction)

3.4 UC-04: Giảm giá theo điểm tích lũy thành viên

Usecase ID: UC-04 – Giảm giá theo điểm tích lũy thành viên	
Tóm tắt: Chức năng tự động xác định mức chiết khấu phần trăm trên tổng giá trị đơn hàng dựa trên số điểm tích lũy và hạng thành viên của khách hàng (Bạc $\geq$ 20 điểm: 5%, Vàng $\geq$ 50 điểm: 10%, Kim Cương $\geq$ 100 điểm: 15%).	
Dòng sự kiện chính	1. Trong quá trình tính toán giá trị đơn hàng, NestJS API truy vấn điểm tích lũy (`loyaltyPoints`) của khách hàng từ bảng `User`. 2. Hệ thống đối chiếu điểm số với các mốc xếp hạng thành viên: • Dưới 20 điểm (Hạng Chuẩn): Chiết khấu 0%. • Từ 20 đến 49 điểm (Hạng Bạc): Chiết khấu 5% tổng tiền món. • Từ 50 đến 99 điểm (Hạng Vàng): Chiết khấu 10% tổng tiền món. • Từ 100 điểm trở lên (Hạng Kim Cương): Chiết khấu 15% tổng tiền món. 3. Hệ thống tính số tiền giảm giá (`discountAmount`) và tính lại tổng tiền thanh toán: `finalAmount = totalAmount - discountAmount`. 4. Hệ thống lưu giá trị chiết khấu vào đơn hàng và phản hồi về cho Next.js UI hiển thị chi tiết số tiền được giảm cho khách hàng.
Dòng sự kiện phụ	• **2a. Khách hàng chưa đạt mốc 20 điểm:** Không áp dụng chiết khấu (`discountAmount = 0`), tổng tiền thanh toán giữ nguyên giá gốc. • **3a. Lỗi truy xuất điểm tích lũy:** Hệ thống mặc định tính theo giá gốc để không gián đoạn quy trình đặt hàng của khách.
Tiền điều kiện	Khách hàng đã đăng nhập và đang trong bước tính giá trị đơn hàng; thỏa mãn điều kiện điểm tích lũy $\geq$ 20.
Hậu điều kiện	• **Thành công:** Tổng số tiền khách hàng cần thanh toán được giảm trừ chính xác theo tỷ lệ chiết khấu của hạng thành viên. • **Thất bại:** Không áp dụng chiết khấu, đơn hàng giữ nguyên giá trị niêm yết ban đầu.

Bảng 4: Bảng đặc tả Use-Case UC-04: Giảm giá theo điểm tích lũy thành viên

3.5 UC-05: Xử lý giao dịch thanh toán

Usecase ID: UC-05 – Xử lý giao dịch thanh toán	
Tóm tắt: Chức năng tiếp nhận yêu cầu thanh toán cho đơn hàng đã tạo, kiểm tra Idempotency, chuyển tiếp giao dịch sang Cổng thanh toán (PayOS / MoMo), cập nhật trạng thái đơn hàng sang `PAID` và kích hoạt tích lũy điểm khi thành công.	
Dòng sự kiện chính	1. Next.js UI gửi yêu cầu `POST /payments` kèm `orderId`, phương thức thanh toán (`WALLET`/`CARD`), `amount` và `idempotencyKey` đến NestJS API. 2. NestJS API kiểm tra trạng thái đơn hàng trong CSDL (phải là `PENDING`). 3. Kích hoạt use-case kiểm tra Idempotency để xác minh giao dịch chưa từng được thực hiện. 4. NestJS API gửi yêu cầu trừ tiền sang API Cổng thanh toán (PayOS / MoMo). 5. Cổng thanh toán xác thực số dư tài khoản/ví, thực hiện trừ tiền và phản hồi trạng thái `SUCCESS` kèm `transactionId`. 6. NestJS API mở Transaction cập nhật trạng thái đơn hàng thành `PAID`, tạo bản ghi `Payment` thành công. 7. Kích hoạt use-case tích lũy điểm thành viên cho khách hàng. 8. NestJS API phản hồi HTTP 200 OK để Next.js UI thông báo thành công và làm rỗng giỏ hàng.
Dòng sự kiện phụ	• **2a. Đơn hàng không tồn tại hoặc đã được thanh toán trước đó:** NestJS API từ chối và trả về HTTP 400 Bad Request. • **5a. Cổng thanh toán từ chối hoặc giao dịch thất bại (Ví không đủ tiền, lỗi thẻ):** Cổng thanh toán phản hồi `FAILED` → Kích hoạt use-case Hoàn trả tồn kho (Stock Rollback), đơn hàng chuyển sang `PAYMENT_FAILED`. • **5b. Cổng thanh toán mất kết nối hoặc quá thời gian (Timeout):** Giữ nguyên trạng thái đơn hàng là `PENDING` để phục vụ cơ chế đối soát sau.
Tiền điều kiện	Đơn hàng đã được tạo thành công ở trạng thái `PENDING`; có chuỗi `idempotencyKey` duy nhất.
Hậu điều kiện	• **Thành công:** Đơn hàng chuyển sang `PAID`, bản ghi thanh toán được lưu trữ, điểm thưởng được cộng cho khách hàng. • **Thất bại:** Đơn hàng chuyển sang `PAYMENT_FAILED`, tồn kho được hoàn trả lại đầy đủ, khách hàng nhận thông báo thử lại.

Bảng 5: Bảng đặc tả Use-Case UC-05: Xử lý giao dịch thanh toán

3.6 UC-06: Kiểm tra Idempotency (Chống trừ tiền trùng)

Usecase ID: UC-06 – Kiểm tra Idempotency (Chống trừ tiền trùng)	
Tóm tắt: Chức năng kiểm tra tính duy nhất của mã khóa Idempotency trong CSDL trước khi thực hiện giao dịch tài chính, ngăn chặn tình trạng khách hàng bị trừ tiền hai lần khi mạng lag hoặc bấm gửi nhiều lần.	
Dòng sự kiện chính	1. PaymentsService tiếp nhận chuỗi `idempotencyKey` từ request thanh toán của Client. 2. Hệ thống thực hiện truy vấn bảng `Payment` trong PostgreSQL theo trường `idempotencyKey`. 3. Nếu khóa **chưa** tồn tại trong CSDL: Hệ thống ghi nhận khóa này và cho phép tiếp tục quy trình gọi sang Cổng thanh toán.
Dòng sự kiện phụ	• **3a. Khóa đã tồn tại với trạng thái `COMPLETED`:** 1. Hệ thống nhận diện đây là yêu cầu gửi lại (retry do mạng lag). 2. Hệ thống lập tức ngắt luồng gọi sang Cổng thanh toán (chặn hoàn toàn việc trừ tiền lần 2). 3. Hệ thống lấy dữ liệu kết quả thanh toán trước đó trong CSDL và phản hồi ngay HTTP 200 OK kèm thông tin giao dịch cũ. • **3b. Khóa đã tồn tại với trạng thái `PROCESSING`:** Hệ thống trả về HTTP 409 Conflict thông báo giao dịch đang được xử lý, đề nghị khách hàng chờ kết quả.
Tiền điều kiện	Yêu cầu thanh toán gửi đến hệ thống có đính kèm tham số `idempotencyKey`.
Hậu điều kiện	• **Thành công:** Đảm bảo giao dịch chỉ được trừ tiền duy nhất một lần trên Cổng thanh toán bất kể số lần Client gửi lại request. • **Thất bại:** Không có giao dịch trùng lặp nào được tạo trong hệ thống.

Bảng 6: Bảng đặc tả Use-Case UC-06: Kiểm tra Idempotency (Chống trừ tiền trùng)

3.7 UC-07: Hoàn trả tồn kho (Stock Rollback)

Usecase ID: UC-07 – Hoàn trả tồn kho (Stock Rollback)	
Tóm tắt: Chức năng tự động cộng bù lại số lượng tồn kho của các sản phẩm đã tạm trừ trước đó khi giao dịch thanh toán bị từ chối hoặc thất bại, giải phóng hàng tồn ảo cho các khách hàng khác đặt mua.	
Dòng sự kiện chính	1. Khi nhận tín hiệu thanh toán thất bại (`FAILED`) từ Cổng thanh toán hoặc lỗi hệ thống: 2. NestJS API mở Transaction nguyên tử trong PostgreSQL. 3. Cập nhật trạng thái của đơn hàng `Order` sang `PAYMENT_FAILED`. 4. Hệ thống truy vấn danh sách các món và số lượng mua tương ứng từ bảng `OrderItem`. 5. Với từng sản phẩm, hệ thống thực hiện hoàn kho: `stock = stock + quantity`, đồng thời tăng `version = version + 1`. 6. Hệ thống commit Transaction hoàn kho và ghi log kiểm toán sự kiện (Audit Log: Stock Rollback Completed). 7. Phản hồi thông báo lỗi về Next.js UI để khách hàng có thể chọn phương thức thanh toán khác hoặc đặt lại.
Dòng sự kiện phụ	• **4a. Không tìm thấy chi tiết món trong đơn hàng:** Hệ thống phát tín hiệu cảnh báo khẩn cấp (Critical Log) để quản trị viên kiểm tra dữ liệu thủ công.
Tiền điều kiện	Đơn hàng đã trừ kho trước đó nhưng giao dịch thanh toán bị hủy, từ chối hoặc thất bại.
Hậu điều kiện	• **Thành công:** Toàn bộ số lượng sản phẩm được hoàn trả chính xác vào kho hàng; trạng thái đơn hàng là `PAYMENT_FAILED`. • **Thất bại:** Transaction hoàn kho được tự động thử lại (retry queue) để đảm bảo không bị thất thoát hàng hóa.

Bảng 7: Bảng đặc tả Use-Case UC-07: Hoàn trả tồn kho (Stock Rollback)

3.8 UC-08: Tích lũy điểm thành viên

Usecase ID: UC-08 – Tích lũy điểm thành viên	
Tóm tắt: Chức năng tính toán và tự động cộng điểm thưởng tích lũy vào tài khoản khách hàng sau khi đơn hàng được thanh toán thành công (10.000 VNĐ = 1 điểm).	
Dòng sự kiện chính	1. Sau khi Cổng thanh toán phản hồi `SUCCESS` và đơn hàng chuyển sang `PAID`: 2. NestJS API lấy giá trị thanh toán thực tế (`totalAmount`) của đơn hàng. 3. Hệ thống tính số điểm thưởng tích lũy: `points = floor(totalAmount / 10000)`. 4. NestJS API mở Transaction cộng điểm vào trường `loyaltyPoints` của người dùng trong bảng `User`: `loyaltyPoints = loyaltyPoints + points`. 5. Hệ thống ghi lại bản ghi lịch sử cộng điểm vào bảng lưu vết (Loyalty History). 6. Phản hồi số điểm mới tích lũy về Next.js UI để hiển thị thông báo chúc mừng trên màn hình thành công.
Dòng sự kiện phụ	• **3a. Tổng tiền đơn hàng dưới 10.000 VNĐ:** Số điểm tích lũy cộng thêm là 0 điểm. • **4a. Gặp lỗi khi cộng điểm vào CSDL:** Không làm gián đoạn trạng thái đơn hàng (đơn hàng vẫn là `PAID`), tác vụ cộng điểm được đẩy vào hàng đợi retry để xử lý lại.
Tiền điều kiện	Đơn hàng đã được thanh toán thành công và chuyển sang trạng thái `PAID`.
Hậu điều kiện	• **Thành công:** Điểm thưởng của khách hàng được cập nhật tăng thêm tương ứng, khách hàng có thể dùng điểm để nâng hạng và nhận chiết khấu cho các đơn hàng tiếp theo. • **Thất bại:** Không ảnh hưởng đến đơn hàng đã thanh toán thành công.

Bảng 8: Bảng đặc tả Use-Case UC-08: Tích lũy điểm thành viên

4 Phạm vi hệ thống: Hệ thống Làm gì và Không làm gì

Hệ thống LÀM GÌ (In-Scope System Behaviors)	**Hệ thống KHÔNG LÀM GÌ (Out-of-Scope)**
Xác thực danh tính người dùng bằng JWT Token trước khi cho phép đặt hàng và thanh toán.	**Không** hỗ trợ thanh toán nặc danh (guest checkout) thiếu tài khoản định danh.
Kiểm soát tồn kho tức thời bằng Khóa lạc quan (Optimistic Concurrency Control qua trường `version`) ngăn chặn bán âm kho (Overselling).	**Không** khóa giữ hàng tồn kho vĩnh viễn trong giỏ khi khách chưa tiến hành bấm thanh toán.
Thực thi toàn bộ quy trình tạo đơn, chi tiết món và trừ kho trong một Atomic Transaction (`$transaction`) duy nhất.	**Không** tách rời việc trừ kho và tạo đơn thành các bước riêng rẽ ngoài transaction.
Chống trừ tiền trùng lặp thông qua cơ chế `idempotencyKey`. Nếu gửi lại cùng một key, trả về kết quả cũ mà không trừ tiền lần 2.	**Không** tích hợp cổng thanh toán thực tế phức tạp (chỉ dùng Payment Gateway mô phỏng chuẩn giao thức).
Tự động tính và cộng điểm tích lũy thành viên (`loyaltyPoints`) ngay khi thanh toán thành công.	**Không** xử lý phân hệ vận chuyển giao hàng tận nhà (chỉ nhận trực tiếp tại quầy đồ uống).

Bảng 9: Bảng xác định phạm vi hành vi hệ thống phục vụ nghiệm thu

5 Mô tả các luồng nghiệp vụ chi tiết

5.1 Luồng chính (Main Success Scenario)

Luồng chính gồm 8 giai đoạn phân định rõ giữa các thành phần kiến trúc:

1. **Khách hàng xác nhận giỏ hàng:** Khách hàng kiểm tra các sản phẩm, kích cỡ và topping trong giỏ hàng, sau đó nhấn nút **"Tiến hành đặt hàng & Thanh toán"**.

2. **Next.js UI kiểm tra xác thực:** Ứng dụng kiểm tra sự tồn tại của JWT Token trong LocalStorage. Token hợp lệ, UI chuyển hướng sang màn hình Checkout hiển thị tổng tiền cần thanh toán.

3. **Khách chọn phương thức & Gửi lệnh:** Khách hàng chọn phương thức Ví điện tử (`WALLET`) và nhấn nút **"Xác nhận thanh toán"**.

4. **Next.js UI khởi tạo đơn:** Next.js UI sinh mã UUID ngẫu nhiên cho trường `idempotencyKey`, sau đó gửi yêu cầu HTTP `POST /orders` kèm Bearer JWT Token và danh sách các món.

5. **NestJS API kiểm tra & Mở Transaction:**

   - `AuthGuard` xác thực chữ ký JWT và trích xuất `userId`.
   - Mở Prisma Database Transaction (`$transaction`): Kiểm tra tồn kho sản phẩm (`stock >= qty`) và phiên bản `version`.
   - Trừ tồn kho và tăng `version = version + 1`.

- Tạo bản ghi `Order` với trạng thái `PENDING` và các bản ghi `OrderItem`, `OrderItemTopping`.
- Commit transaction và phản hồi HTTP `201 Created` chứa `orderId` và `totalAmount`.

6. **Gọi xử lý thanh toán:** Next.js UI gửi tiếp yêu cầu `POST /payments` kèm `orderId`, `method: 'WALLET'`, `amount` và `idempotencyKey`.

7. **NestJS API & Payment Service:**

   - Kiểm tra `idempotencyKey` chưa từng tồn tại trong CSDL.
   - Gọi Cổng thanh toán mô phỏng. Cổng thanh toán kiểm tra số dư và phản hồi `{ status: 'SUCCESS', transactionId: 'TXN_...' }`.
   - NestJS API cập nhật trạng thái đơn hàng sang `PAID`, tạo bản ghi `Payment`, và cộng điểm thưởng `loyaltyPoints` cho khách hàng.

8. **Hoàn tất giao dịch:** NestJS API trả về HTTP `200 OK`. Next.js UI gọi hàm `clearCart()` làm rỗng giỏ hàng, hiển thị Toast thông báo thành công và chuyển hướng đến màn hình theo dõi đơn hàng (`/orders/:id`).

5.2 Các luồng thay thế (Alternative Flows)

- **A1 – Thay đổi phương thức thanh toán:** Tại Bước 3, khách hàng có thể chuyển đổi giữa Ví điện tử (`VietQR`) và (`Momo`).
- **A2 – Xử lý gửi lại với IMomodempotency Key (Chống trừ tiền trùng):** Trường hợp mạng lag khách hàng bấm 2 lần liên tiếp, hoặc Client retry gửi lại cùng một `idempotencyKey`. Tại Bước 7, NestJS API phát hiện key đã tồn tại với trạng thái `COMPLETED` → Hệ thống không gọi cổng thanh toán lần 2 và không trừ tiền lần 2, mà trả về ngay dữ liệu kết quả cũ kèm HTTP `200 OK`.

5.3 Các luồng ngoại lệ (Exception Flows)

- **E1 – Giỏ hàng rỗng:** Khách hàng truy cập thẳng vào trang Checkout khi chưa chọn món → Next.js UI chặn lại, hiển thị cảnh báo và chuyển hướng về trang `/menu`.
- **E2 – Chưa đăng nhập hoặc Token hết hạn (HTTP 401):** NestJS API từ chối request → UI lưu tạm giỏ hàng vào bộ nhớ và điều hướng sang trang đăng nhập (`/login?redirect=/checkout`).
- **E3 – Hết hàng hoặc xung đột tồn kho (HTTP 400/409):** Khi tạo đơn, nếu sản phẩm hết hàng hoặc sai phiên bản `version` → Transaction tự động Rollback, không có đơn hàng nào được tạo, UI hiển thị thông báo lỗi màu đỏ.
- **E4 – Thanh toán bị từ chối (HTTP 402/400):** Cổng thanh toán báo `FAILED` (số dư không đủ, thẻ khóa) → Đơn hàng chuyển sang `PAYMENT_FAILED`, hệ thống mở transaction hoàn lại số lượng tồn kho (Stock rollback). UI cho phép khách hàng đổi phương thức thanh toán để thử lại.
- **E5 – Cổng thanh toán mất kết nối (HTTP 504 Gateway Timeout):** Hệ thống quá thời gian phản hồi (timeout) → Giữ trạng thái đơn là `PENDING` chờ kiểm tra đối soát, thông báo người dùng kiểm tra lịch sử đơn.

6 Tiêu chí chấp nhận (Acceptance Criteria)

Mã Tiêu Chí	Kịch bản kiểm chứng chi tiết (Given – When – Then)
AC-INT-01	**Tạo đơn hàng trong Database Transaction thành công:** *Given:* Khách hàng có token JWT hợp lệ và món A có `stock = 10`, `version = 1`. *When:* Gửi `POST /orders` đặt mua số lượng 2 món A. *Then:* Trả về HTTP `201 Created`, tạo đơn `PENDING`, món A còn `stock = 8`, `version = 2`.
AC-INT-02	**Từ chối tạo đơn khi vượt quá tồn kho (Optimistic Lock):** *Given:* Món B chỉ còn `stock = 1`. *When:* Gửi `POST /orders` yêu cầu số lượng 2. *Then:* Trả về HTTP `400 Bad Request`, Transaction rollback, kho giữ nguyên `stock = 1`.
AC-INT-03	**Xử lý Idempotency Key an toàn chống trùng giao dịch:** *Given:* Đơn hàng #101 đã thanh toán thành công với mã `idempotencyKey = "uuid-key-01"`. *When:* Gửi lại request `POST /payments` lần 2 với cùng `idempotencyKey = "uuid-key-01"`. *Then:* Trả về HTTP `200 OK` với dữ liệu cũ, không gọi trừ tiền lần 2, không tạo Payment mới.
AC-INT-04	**Tự động cộng điểm thưởng thành viên:** *Given:* Tài khoản khách hàng có 10 điểm tích lũy; đơn hàng giá trị 50.000đ. *When:* Thanh toán thành công chuyển trạng thái đơn sang `PAID`. *Then:* Điểm `loyaltyPoints` của người dùng tăng lên đúng 15 điểm.
AC-E2E-01	**Kiểm thử luồng mua hàng và thanh toán trên giao diện người dùng (E2E):** *Given:* Khách mở trình duyệt Next.js, đăng nhập và thêm món vào giỏ hàng. *When:* Khách bấm Checkout → Chọn Ví điện tử → Xác nhận thanh toán. *Then:* Hiển thị Spinner tải, thông báo Toast thành công, giỏ hàng về rỗng, chuyển sang trang đơn hàng.

Bảng 10: Bảng Tiêu chí chấp nhận có thể kiểm chứng (Acceptance Criteria)

7 Bảng ánh xạ kỹ thuật phục vụ vẽ Sequence Diagram (Task 2.2)

Bảng dưới đây đóng vai trò cấu nối trực tiếp giúp hiện thực hóa sơ đồ tuần tự (Sequence Diagram) ở Task 2.2 với 5 Lifelines quy định:

Bước	Lifeline Gửi	Lifeline Nhận	Thao tác / Endpoint API	Dữ liệu truyền tải / Ý nghĩa
1	Customer	Next.js UI	Click Button	`clickButton("Thanh toán")`
2	Next.js UI	NestJS API	`POST /orders`	Header: Bearer JWT; Body: `{ items: [...] }`
3	NestJS API	NestJS API	Token Verification	`jwtService.verify(token)` trích xuất userId
4	NestJS API	Database	Prisma `$transaction`	Kiểm tra stock/version, trừ kho, tạo Order PENDING
5	Database	NestJS API	SQL Return	Trả về `orderId = 101, status: "PENDING"`
6	NestJS API	Next.js UI	HTTP 201 Created	Phản hồi tạo đơn thành công kèm orderId
7	Next.js UI	NestJS API	`POST /payments`	Body: `{ orderId, method, idempotencyKey }`
8	NestJS API	Database	SQL Query	Kiểm tra `idempotencyKey` chống trừ tiền trùng
9	NestJS API	Payment Service	HTTP POST /mock-pay	Gọi cổng thanh toán mô phỏng
10 (alt 1)	Payment Service	NestJS API	HTTP 200 OK	Phản hồi thanh toán thành công `SUCCESS`
11 (alt 1)	NestJS API	Database	Prisma `$transaction`	Lưu Payment, Order sang `PAID`, cộng điểm thưởng
12 (alt 2)	Payment Service	NestJS API	HTTP 400 Bad Request	Phản hồi thanh toán thất bại `FAILED`
13 (alt 2)	NestJS API	Database	Prisma `$transaction`	Cập nhật `PAYMENT_FAILED`, rollback hoàn kho
14	NestJS API	Next.js UI	HTTP 200 / 400	Trả về kết quả giao dịch cuối cùng
15	Next.js UI	Customer	UI Render	Gọi `clearCart()`, thông báo Toast, mở đơn hàng

Bảng 11: Các bước giao tiếp tương ứng cho Sequence Diagram Task 2.2